Tutoriat LMC

Vlăduț Ghiță, Raluca Ionete, Voaideș Robert

March 24, 2026

Contents

1 Introducere în LP și Prolog **2**
1.1 Administrativ 2
1.2 De ce Logică? 2
1.3 Logica Propozițională - intro 3
1.3.1 Principiul inducției pe formule 4
1.3.2 Principiul recursiei pe formule 5
1.4 Semantica LP 6
1.4.1 Evaluări 6
1.4.2 Modele. Tautologii 7
1.5 Prolog Basics 9

Chapter 1

Introducere în LP și Prolog

1.1 Administrativ

- Cursurile d-lui Leuștean sunt foarte bine scrise și chiar merită citite pentru a învăța la materia asta. Le găsiți pe toate pe GitHub. În plus, la examen **o să fie nevoie** să folosiți propoziții din curs (care sunt numerotate), și să le citați, așa că vă ajută să știți cam care este structura cursului. Nu o să reluăm teoria pe care o prezintă în curs **decât** dacă ne cereți asta.
- Seminariile sunt destul de complexe, acoperă ok materia, iar soluțiile sunt postate pe site, așa că noi o să încercăm să lucrăm la tutoriat exerciții extra.
- Puteți să ne scrieți oricând în privat/pe grup cu întrebări/sugestii/idei, iar noi o să încercăm să vă ajutăm și să le punem în practică. De asemenea, puteți să ne sugerați ce ați refera să lucrăm la tutoriat.
- O să publicăm pe GitHub tutoriatele săptămânal și o să anunțăm, prin README, ce o să facem la următorul tutoriat. De asemenea, plănuim să creăm liste de exerciții, la care să publicăm și rezolvările. Dacă o să doriți să ne uităm pe rezolvările voaste, puteți să ne ziceți.

1.2 De ce Logică?

Este foarte normal să vă întrebați de ce e nevoie să studiați logica ca studenți la info, având în vedere că e aproape matematică pură, care nu pare să aibă foarte mare aplicabilitate în viața reală.
Well, sunt mai multe motive.

Logica e baza informaticii

Aproape toți oamenii care au fondat informatica erau logicieni. Tot ce înseamnă informatică formală se poate traduce în logică, literally.

Asta se traduce în două lucruri:

- nu o să scăpați de logică pentru încă ceva timp, în anul II aveți două materii care se construiesc pe logică și la care o să vă ajute foarte mult să aveți bazele bine stabilite.
- felul foarte formal de a gândi care este promovat de logică vă va ajuta foarte mult în a deveni programatori mai buni; o să ajungeți să gândiți mult mai mult ce scrieți, atât în matematică cât și în programare, pentru a fi mult mai riguros și mai logic exprimat, ceea ce o să vă ajute mai ales atunci când aveți lucruri mai complicate de făcut.

Logica vă deschide drumul și către o ramură destul de nișată a informaticii, dar care este foarte bine plătită și foarte AI-proof, anume verificarea formală, care, pe scurt, presupune demonstrarea matematică a corectitudinii unui cod, lucru care este esențial în sisteme high-risk (procesoare, sisteme medicale, rachete etc.) Facultatea noastră este foarte bogată în logicieni, se țin foarte des seminarii de logică la ILDS sau LOS, așa că dacă vreți să faceți cercetare în timpul facultății sau să vă specializați pe partea de verificare formală sau alte tipuri de logică, chiar aveți cu cine.

1.3 Logica Propozițională - intro

Formulele stau la baza Logicii Propoziționale și sunt principalul lucru cu care o să lucrați capitolul ăsta. Cursul menționează că ele sunt definite inductiv:

Definition 1 (Formule). Formulele sunt definite astfel:

(1) Orice variabilă propozițională este formulă. (Adică $\forall v \in V$, v este formulă)

(2) Dacă φ este formulă, atunci $\neg\varphi$ este formulă

(3) Dacă φ, ψ sunt formule, atunci $\psi \to \varphi$ este formulă

Dar de ce este inductiv? Până acum, ați folosit doar inducția pe numere naturale, care știți cum funcționa. Aveam un caz de bază $P(0)$ și, presupunând că predicatul era adevărat pentru un n natural, construiam următorul caz, $n+1$, și demonstram corectitudinea.

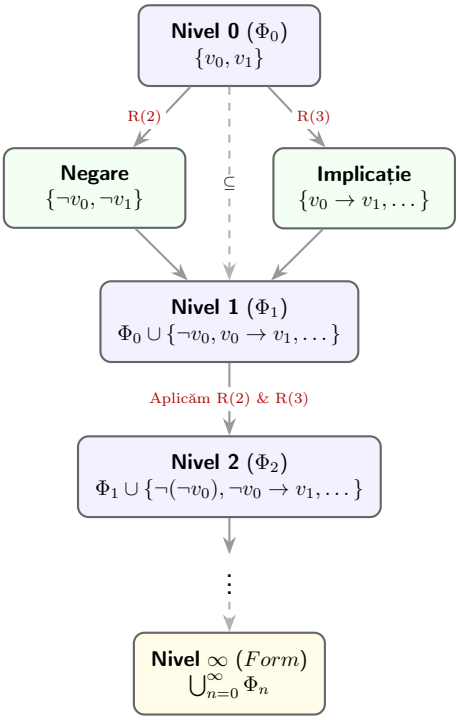
Practic, inducția reprezintă metoda de a pleca de la ceva particular și de a generaliza incremental. Pentru numere naturale, pasul incremental este simplu, anume adunarea.

Noi putem extinde metoda asta, astfel, și la formule. Avem cazul particular, pe care îl definim a fi formulă, adică variabilele v. De aici, incremental, construim toate celelalte formule. În loc de un singur pas incremental, avem doi:

$\neg$ putem nega formula noastră curentă pentru a obține o nouă formulă

$\to$ atunci când avem două formule, o nouă formulă este definită prin implicație

Această metodă se numește **inducție structurală**.
În următoarea figură puteți vedea cum sunt construite formulele din doar două variabile.



1.3.1 Principiul inducției pe formule

Același principiu poate fi a aplicat pentru a demonstra că orice formulă respectă o proprietate $\mathbf{P}$ (e.g., orice formulă are mai $\leq$ conectori logici decât variabile, orice formulă este bine parantezată, etc.).

Proposition 1. Fie $\mathbf{P}$ o proprietate. Presupunem că:

(0) Orice variabilă v are proprietatea $\mathbf{P}$.

(1) Pentru orice formulă φ, dacă φ are proprietatea $\mathbf{P}$, atunci $\neg\varphi$ are proprietatea $\mathbf{P}$.

(2) Pentru orice formule φ, ψ, dacă acestea au proprietatea $\mathbf{P}$, atunci $\varphi \to \psi$ are proprietatea $\mathbf{P}$.

Atunci orice formulă φ are proprietatea $\mathbf{P}$.

Example. Vrem să demonstrăm că orice formulă φ este parantezată corect. Formal: orice formulă φ are proprietatea $\mathbf{P}(\varphi) : st(\varphi) = dr(\varphi)$, unde $st(\varphi)$ este numărul de paranteze "(", iar $dr(\varphi)$ este numărul de paranteze ")".

Proof. Demonstrăm proprietatea $P(\varphi)$ prin inducție structurală după construcția formulei φ.

Cazul de bază: $\varphi = v$, unde $v \in V$ este o variabilă propozițională. Conform definiției sintaxei, variabilele nu conțin paranteze.

$$st(v) = 0 \text{ și } dr(v) = 0 \Rightarrow st(v) = dr(v)$$

Deci $P(v)$ este adevărată.

Pasul inductiv: Presupunem că proprietatea este adevărată pentru formulele ψ și χ (Ipoteza Inductivă). Deci $st(\psi) = dr(\psi)$ și $st(\chi) = dr(\chi)$.

Demonstrăm pentru regulile de construcție:

- **Cazul** $\varphi = (\neg\psi)$: Calculăm numărul de paranteze folosind definiția recursivă:

$$st(\varphi) = st(\psi) + 1$$
$$dr(\varphi) = dr(\psi) + 1$$

  Din Ipoteza Inductivă știm că $st(\psi) = dr(\psi)$. Adunând 1 în ambele părți, obținem $st(\varphi) = dr(\varphi)$.

- **Cazul** $\varphi = (\psi \to \chi)$: Calculăm numărul de paranteze:

$$st(\varphi) = 1 + st(\psi) + st(\chi)$$
$$dr(\varphi) = dr(\psi) + dr(\chi) + 1$$

  Înlocuind din Ipoteza Inductivă ($st \to dr$), avem:

$$st(\varphi) = 1 + dr(\psi) + dr(\chi)$$

  Datorită comutativității adunării, $1 + dr(\psi) + dr(\chi) = dr(\psi) + dr(\chi) + 1$, deci $st(\varphi) = dr(\varphi)$.

Conform principiului inducției structurale, proprietatea $P(\varphi)$ este adevărată pentru orice formulă φ. □

1.3.2 Principiul recursiei pe formule

Dacă inducția ne ajută să demonstrăm proprietăți, principiul recursiei pe formule este folosit pentru a da **definiții recursive** riguroase diverselor funcții asociate formulelor.

Proposition 2 (Principiul recursiei pe formule)**.** Fie A o mulțime și funcțiile:

$$G_0 : V \to A, \quad G_\neg : A \to A, \quad G_\to : A \times A \to A$$

Atunci există o **unică** funcție $F : Form \to A$ care satisface următoarele

proprietăți:

(R0) $F(v) = G_0(v)$ pentru orice variabilă $v \in V$.

(R1) $F(\neg\varphi) = G_\neg(F(\varphi))$ pentru orice formulă φ.

(R2) $F(\varphi \to \psi) = G_\to(F(\varphi), F(\psi))$ pentru orice formule φ, ψ.

Example. Vrem să definim o funcție $var : Form \to \mathcal{P}(V)$ care extrage mulțimea tuturor variabilelor propoziționale ce apar într-o formulă φ (unde $\mathcal{P}(V)$ este mulțimea părților lui V). O definiție recursivă a lui var este următoarea:

$$\begin{aligned} var(v) &= \{v\} && \text{pentru orice variabilă } v \\ var(\neg\varphi) &= var(\varphi) && \text{pentru orice formulă } \varphi \\ var(\varphi \to \psi) &= var(\varphi) \cup var(\psi) && \text{pentru orice formule } \varphi, \psi \end{aligned}$$

În acest caz, maparea pe principiul general se face astfel:

- Mulțimea A este $\mathcal{P}(V)$.
- $G_0 : V \to \mathcal{P}(V)$, definită prin $G_0(v) = \{v\}$.
- $G_\neg : \mathcal{P}(V) \to \mathcal{P}(V)$, definită prin $G_\neg(X) = X$ (negația nu modifică variabilele existente).
- $G_\to : \mathcal{P}(V) \times \mathcal{P}(V) \to \mathcal{P}(V)$, definită prin $G_\to(X, Y) = X \cup Y$ (reunim variabilele din ambele subformule).

1.4 Semantica LP

În acest capitol se studiază semantica, adică, informal, **ce înseamnă o formulă/cum îi dăm sens**. Acest lucru se face prin intermediul **evaluărilor** (sau interpretărilor, dar mai rar), care sunt funcții ce **dau valoare variabilelor propoziționale**.

1.4.1 Evaluări

O evaluarea este orice funcție:

$$e : V \to \{0, 1\}$$

Cu ajutorul recursiei pe formule este foarte ușor să extindem de la evaluări (care sunt pe variabile), la o funcție care interpretează formule:

Theorem 1. Pentru orice evaluare $e : V \to \{0, 1\}$ există o unică funcție (vezi recursia pe formule):

$$e^+ : Form \to \{0,1\}$$

care verifică următoarele proprietăți

(1) $e^+(v) = e(v), \quad \forall v \in V$

(2) $e^+(\neg\varphi) = \neg e^+(\varphi), \quad \forall \varphi \in Form$

(3) $e^+(\varphi \to \psi) = e^+(\varphi) \to e^+(\psi), \quad \forall \varphi, \psi \in Form$

Două cum puteți vedea, conectorii logici $(\neg, \to)$ sunt îngroșați atunci când sunt scoși din e^+. Acest lucru este pentru a arăta că aceștia nu mai sunt simboluri, în cadrul formulelor, ci **operații** pe $\{0,1\}$, cu tabele de adevăr știute.

Proposition 3. Dacă două evaluări sunt identice pentru toate variabilele dintr-o formulă φ, atunci ele vor da aceeași evaluare a formulei.

$$e_1(v) = e_2(v)\ \forall v \in Var(\varphi) \Rightarrow e_1^+(\varphi) = e_2^+(\varphi)$$

Propoziția 3 este utilă prin faptul că, atunci când evaluăm formule, putem să ne restrângem doar la variabilele din formulele pe care le evaluăm. Acest lucru se traduce în **metoda tabelului**, în care fiecare linie a tabelului este o evaluare diferită a variabilelor din $Var(\varphi)$.

1.4.2 Modele. Tautologii

Definition 2. Fie o evaluare $e : V \to \{0,1\}$.

Model: e este **model** pentru o formulă φ dacă $e^+(\varphi) = 1$. Notăm $e \models \varphi$.

Satisfiabilitate: O formulă φ se numește **satisfiabilă** dacă există cel puțin o evaluare e care este model pentru φ (adică $\exists e$ a.î. $e \models \varphi$).

Nesatisfiabilitate: O formulă φ este **nesatisfiabilă** (sau **contradictorie**) dacă nu admite niciun model (adică pentru orice evaluare e, $e^+(\varphi) = 0$). Notăm uneori prin $\varphi \models \bot$.

Tautologie: O formulă φ este **tautologie** dacă orice evaluare este model pentru ea (adică $\forall e, e \models \varphi$). Notăm acest lucru prin $\models \varphi$.

Notation. Mulțimea tuturor modelelor unei formule φ se notează cu $Mod(\varphi)$.

$$Mod(\varphi) = \{e : V \to \{0,1\} \mid e \models \varphi\}$$

Toți termenii definiți în 2 sunt foarte importanți și trebuie reținuți, pentru că o să tot lucrați cu ei.

Example. Formula $p \vee \neg p$ este o **tautologie**, deoarece indiferent de valoarea lui p, formula va fi adevărată. În schimb, $p \wedge \neg p$ este o **contradicție**, deoarece nicio evaluare nu o poate face adevărată.

Proposition 4 (Legile lui De Morgan). Pentru orice formule $\varphi, \psi \in Form$, au loc următoarele echivalențe logice:

$$\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi \tag{10}$$
$$\neg(\varphi \wedge \psi) \sim \neg\varphi \vee \neg\psi \tag{11}$$

Demonstrație:

Vom demonstra prima lege: $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$. Demonstrația celei de-a doua legi se realizează în mod absolut analog.

(i) Metoda tabelului de adevăr

Fie $e : V \to \{0, 1\}$ o evaluare oarecare. Construim tabelul de adevăr pentru a evalua extensia e^+ pentru ambele formule. Coloanele de la stânga liniei verticale reprezintă valorile posibile pentru $e^+(\varphi)$ și $e^+(\psi)$, iar în dreapta evaluăm pas cu pas subformulele.

φ	ψ	$\varphi \vee \psi$	$\neg(\varphi \vee \psi)$	$\neg\varphi$	$\neg\psi$	$\neg\varphi \wedge \neg\psi$
1	1	1	0	0	0	0
1	0	1	0	0	1	0
0	1	1	0	1	0	0
0	0	0	1	1	1	1

Analizând tabelul, observăm că pentru toate cele 4 cazuri posibile ale unei evaluări e, valoarea sub coloana $\neg(\varphi \vee \psi)$ este identică cu valoarea sub coloana $\neg\varphi \wedge \neg\psi$. Astfel, $e^+(\neg(\varphi \vee \psi)) = e^+(\neg\varphi \wedge \neg\psi)$ pentru orice evaluare e, ceea ce înseamnă că au exact aceleași modele. Prin urmare, $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$.

(ii) Metoda prin raționament semantic (formal)

Raționăm direct. Pentru o evaluare arbitrară $e : V \to \{0, 1\}$, observăm că:

$$e^+(\neg(\varphi \vee \psi)) = \neg(e^+(\varphi) \vee e^+(\psi)),$$
$$e^+(\neg\varphi \wedge \neg\psi) = \neg e^+(\varphi) \wedge \neg e^+(\psi).$$

Avem cazurile:

(a) $e^+(\varphi) = 0$. Atunci

$$\neg(e^+(\varphi) \vee e^+(\psi)) = \neg(0 \vee e^+(\psi)) = \neg e^+(\psi),$$
$$\neg e^+(\varphi) \wedge \neg e^+(\psi) = \neg 0 \wedge \neg e^+(\psi) = 1 \wedge \neg e^+(\psi) = \neg e^+(\psi).$$

(b) $e^+(\varphi) = 1$. Atunci

$$\neg(e^+(\varphi) \vee e^+(\psi)) = \neg(1 \vee e^+(\psi)) = \neg 1 = 0,$$
$$\neg e^+(\varphi) \wedge \neg e^+(\psi) = \neg 1 \wedge \neg e^+(\psi) = 0 \wedge \neg e^+(\psi) = 0.$$

Deoarece în ambele cazuri rezultatele evaluărilor coincid, obținem că $e^+(\neg(\varphi \vee \psi)) = e^+(\neg\varphi \wedge \neg\psi)$ pentru orice evaluare e. Rezultă $Mod(\neg(\varphi \vee \psi)) = Mod(\neg\varphi \wedge \neg\psi)$, deci $\neg(\varphi \vee \psi) \sim \neg\varphi \wedge \neg\psi$. $\square$

1.5 Prolog Basics

Prologul are la baza **rezolutia**, despre care o sa invatati mai tarziu in curs, dar pe scurt, se bazeaza pe a ajunge la un adevar. Noi cand scriem cod in prolog de fapt scriem la fiecare termen **un predicat** (o functie care returneaza bool), si la chestii mai complicate prologul practic cauta o cale catre un adevar prin predicatele pe care le definim noi.

> **Note** (Litere mari vs mici). Se face distinctie intre parametrii cu litere mari si litere mici:
>
> Cele cu litere mici se refera la lucruri concrete (numere, stringuri, etc.).
>
> Cele cu litere mari sunt variabile si ele de fapt pot reprezenta orice termen in spate, fie numere, stringuri, liste etc.
>
> In linia de comanda cand rulam programul daca dam variabile ca parametrii prolog ii va interpreta ca necunoscute si incearca sa gaseasca valori care respecta regulile pentru acele variabile.

Mai jos aveti un exemplu de niste predicate simple (fapte) si un predicat definit prin reguli:

Listing 1.1: Family relations in Prolog

```
% Cazurile de baza (oarecum ca la o functie recursiva)
parent(albert, bob).
parent(albert, betsy).
parent(albert, bill).

parent(alice, bob).
parent(alice, betsy).
parent(alice, bill).

% Reguli extra, prolog va incerca sa ajunga la un adevar pe care l-am definit mai sus
% Separarea prin virgula inseamna 'si' intre toate predicatele enumerate
sibling(X, Y) :-
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

Este important sa distingem intre operatorii =, ==, 'is':

- Operatorul = de fapt este operatorul de unificare, prolog va incerca sa unifice cei doi termeni cand face backtracking sa caute solutii.
- Operatorul == comparare stricta, nu incearca sa unifice termenii.
- Operatorul 'is' evalueaza expresii matematice.

Exemple:

Listing 1.2: Distinctie operatorii in prolog

```
?- f(X, 2) = f(1, Y).
X = 1,
Y = 2.

?- f(X, 2) == f(1, Y).
false. % Aici se face comparare fara sa incerce sa unifice termenii X si Y

?- X = 2 + 3.
X = 2+3.

?- X is 2 + 3.
X = 5.

?- 5 == 2 + 3
false. % Aici termenul '5' nu este egal cu termenul '2 + 3'
```

Liste in Prolog

Listele arata vizual ca si in python: `[1, 2, abc, 4, 5]` dar modul in care lucram cu ele este diferit. In prolog ne folosim de o sintaxa cu Head si Tail:

Listing 1.3: Liste in prolog

```
?- [H|T] = [1, 2, 3, 4, 5].
H = 1,
T = [2, 3, 4, 5].

?- [A, B, C|T] = [1, 2, 3, 4, 5].
A = 1,
B = 2,
C = 3,
T = [4, 5].

?- [_|T] = [1, 2, 3, 4, 5].
T = [2, 3, 4, 5]. % _ este discard character, cand nu iti pasa de acel parametru
```

Exercitiu de exemplu (P07 din P99):

Se da o lista sub forma `[1, [2, [3, 4], 5], 6]` si trebuie sa o transformati in `[1, 2, 3, 4, 5, 6]`.

Rezolvare:

Listing 1.4: Exercitiu Flatten List

```
my_flatten([], []). % Cazul de baza

% Cazul cand elementul curent este o lista
my_flatten([H|T], R) :-
    is_list(H),
    my_flatten(H, Fh),
    my_flatten(T, Ft),
    append(Fh, Ft, R).

% Cazul cand elementul curent nu este o lista
my_flatten([H|T], [H|Tr]) :-
    \+ is_list(H),  % Operatorul \+ este operatorul not
    my_flatten(T, Tr).
```

Exercitii propuse

Exercitiul 1

Definiti `my_append/3` echivalent cu append-ul built in.

Rulati si confirmati:

```
?- my_append([1, 2, 3], [4, 5, 6], X)
X = [1, 2, 3, 4, 5, 6].
?- my_append([1, 2, X], [4, 5, 6], [1, 2, 3, 4, 5, 6])
X = 3.
?- my_append([X, 2, Z], [4, 5, 6], [1, 2, 3, 4, Y, 6])
X = 1, Z = 3, Y = 5.
?- my_append(L, [4, 5, 6], [1, 2, 3, 4, 5, 6])
L = [1, 2, 3].
?- my_append(L, [4, 5, 6], [1, 2, 3, 5, 6])
false.
```

Exercitiul 2

Definiti `duplicate/2` care repeta de doua ori elementele dintr-o lista.

Rulati si confirmati:

```
?- duplicate([1, 2, 3], X)
X = [1, 1, 2, 2, 3, 3].
?- duplicate([1, 2, X], [1, 1, 2, 2, 3, 3])
X = 3.
?- duplicate([1, 2, X], [1, 1, 2, 2, 3])
false.
```

Exercitiul 3

Definiti `split/4` care sparge lista primita ca primul parametru la pozitia primita ca al doilea parametru in doua liste (parametrii 3 si 4).

Rulati si confirmati:

```
?- subset([1, 2, 3], X)
X = []; X = [3]; X = [2]; X = [2, 3]; X = [1]; X = [1, 3]; X = [1,
2]; X = [1, 2, 3]
```